

Table of contents

Series 9: Abstract Interpretation I	1
Methods	1
Construction of CFG (Control Flow Graph)	1
Collecting Semantics	2
Iterative Resolution of the Equation System	2
Abstraction by Intervals	2
Abstraction by Signs	2
Reminder	2
Illustrative Examples	3
Example 1: CFG Construction for <code>sillyprogram</code>	3
Example 2: Collecting Semantics for <code>sillyprogram</code>	4
Example 3: Concrete Resolution for <code>a=5, b=3</code>	5
Example 4: Abstraction by Intervals	5
Example 5: Abstraction by Signs	6
Synthesis	6

Series 9: Abstract Interpretation I

Methods

Construction of CFG (Control Flow Graph)

- **Inductive method:** Define a recursive function `CFG(P, ip, rp)` that constructs the control flow graph for a program `P`, with an entry point `ip` and a return point `rp`.
- **Construction rules:**
 - **Assignment:** `CFG(x:=e, ip, rp)` adds a transition labeled `x:=e` from `ip` to `rp`.
 - **Sequence:** `CFG(e1; e2, ip, rp)` creates an intermediate point `e1` and recursively calls `CFG(e1, ip, e1)` and `CFG(e2, e1, rp)`.
 - **Conditional:** `CFG(if c then e1 else e2, ip, rp)` creates two new points `e1` and `e2`, adds transitions from `ip` to `e1` (label `c`) and from `ip` to `e2` (label `¬c`), then recursively calls `CFG(e1, e1, rp)` and `CFG(e2, e2, rp)`.
 - **Loop:** `CFG(while c do e1 done, ip, rp)` creates a point `e1`, adds a transition from `ip` to `e1` (label `c`) and one from `ip` to `rp` (label `¬c`), then calls `CFG(e1, e1, ip)`.

Collecting Semantics

- **System of equations:** Associate with each control point s a variable $V(s)$ representing the set of possible memory states at that point.
- **Equations:** Deduce equations from the CFG structure, using union operations and filtering by conditions.
- **Entry point:** Introduce an initial state st to model parameter initialization.

Iterative Resolution of the Equation System

- **Iteration to fixed point:** Initialize $V(s_0)$ with the initial state, then recursively apply equations until stabilization (or divergence).
- **Termination analysis:** Observe value evolution to conclude about program behavior (termination, infinite loop, error).

Abstraction by Intervals

- **Abstract domain:** Replace concrete sets of values with intervals (e.g., $[2, 6]$ for $\{2, 3, 4, 5, 6\}$).
- **Abstract operations:** Adapt operations (assignment, union, intersection) to work on intervals.
- **Goal:** Simplify analysis by losing precision, but guaranteeing termination and capturing general properties.

Abstraction by Signs

- **Abstract domain:** Represent sets of values by their sign: 0 , 0 , <0 , >0 , $=0$, 0 , (no value), (all values).
- **Abstract operations:** Define operations on signs (e.g., addition, subtraction, comparison).
- **Goal:** Even coarser but faster analysis, to infer simple properties about variables.

Reminder

- **CFG:** Graphical representation of program control flow. Nodes are control points, arcs are transitions labeled by conditions or assignments.
- **Collecting semantics:** Semantics that associates with each control point the set of all possible memory states during execution.

- **Fixed point:** In abstract interpretation, we look for a fixed point of the equation system (a state where $V(s)$ no longer changes after an iteration). The iterative algorithm converges to this fixed point (under certain conditions).
- **Abstraction:** Function that maps a concrete set (e.g., $\{2,3,4\}$) to an element of an abstract domain (e.g., interval $[2,4]$ or sign >0). It must be safe (over-approximative).
- **Abstract operations:** Must over-approximate concrete operations. For example, interval addition $[a,b] + [c,d] = [a+c, b+d]$.

Illustrative Examples

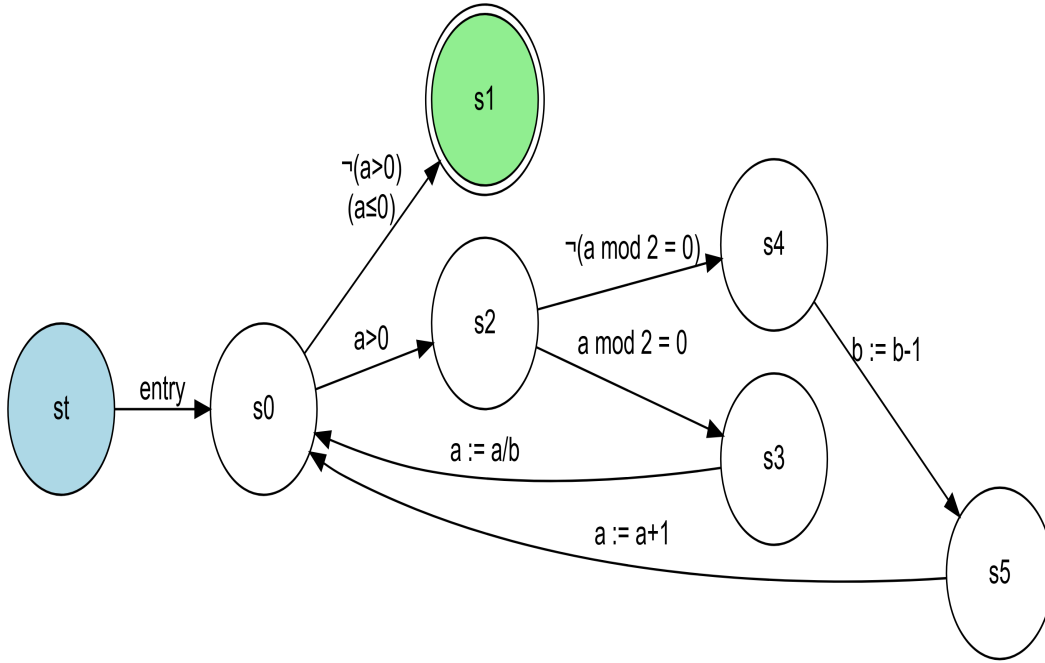
Example 1: CFG Construction for `sillyprogram`

Context: Program `sillyprogram(a,b) = while (a > 0) do if a mod 2 = 0 then a := a/b else b := b-1; a := a+1 endif done`. We want to construct its CFG.

Application: Apply inductive rules.

1. **Loop while:** `CFG(while (a>0) do ... done, ip, rp)`
 - Creates a new point `e1` for the loop body.
 - Adds a transition from `ip` to `e1` labeled `a>0`.
 - Adds a transition from `ip` to `rp` labeled $\neg(a>0)$ (i.e., `a<=0`).
 - Recursive call: `CFG(body, e1, ip)` where `body` is the `if ... endif` instruction.
2. **Conditional if:** `CFG(if a mod 2 = 0 then a:=a/b else b:=b-1; a:=a+1, e1, ip)`
 - Creates two points `e2` (for then) and `e3` (for else).
 - Adds a transition from `e1` to `e2` labeled `a mod 2 = 0`.
 - Adds a transition from `e1` to `e3` labeled $\neg(a \text{ mod } 2 = 0)$.
 - Recursive calls:
 - `CFG(a:=a/b, e2, ip)`
 - `CFG(b:=b-1; a:=a+1, e3, ip)`
3. **Sequence:** `CFG(b:=b-1; a:=a+1, e3, ip)`
 - Creates an intermediate point `e4`.
 - Recursive calls:
 - `CFG(b:=b-1, e3, e4)`
 - `CFG(a:=a+1, e4, ip)`

The resulting CFG has control points `st` (initial), `s0` (loop start), `s1` (loop exit), `s2` (if start), `s3` (then branch), `s4` (else branch), `s5` (after `b:=b-1`). Transitions are labeled as described.



Example 2: Collecting Semantics for sillyprogram

Context: We have the CFG of `sillyprogram`. We want to write the collecting semantics equation system.

Application: Define $V(s)$ for each control point s .

$$\begin{aligned}
 V(st) &= \top \quad (\text{unspecified initial state}) \\
 V(s0) &= V(st)[a := a_0, b := b_0] \cup V(s3)[a := a/b] \cup V(s5)[a := a + 1] \\
 V(s1) &= V(s0) \cap [a \leq 0] \\
 V(s2) &= V(s0) \cap [a > 0] \\
 V(s3) &= V(s2) \cap [a \bmod 2 = 0] \\
 V(s4) &= V(s2) \cap [a \bmod 2 \neq 0] \\
 V(s5) &= V(s4)[b := b - 1]
 \end{aligned}$$

Here, $[\text{cond}]$ denotes the set of states satisfying condition cond . The operation $V(s)[x := \text{expr}]$ means “take all states from $V(s)$, evaluate expr in each state, and assign the resulting value to x ”.

Example 3: Concrete Resolution for $a=5, b=3$

Context: Solve the equation system for initial values $a=5, b=3$.

Application: Iterate manually.

- Initialization: $V(s_0) = \{a:\{5\}, b:\{3\}\}$
- First iteration (loop):
 - Via s_2 (since $a>0$): $V(s_2) = \{a:\{5\}, b:\{3\}\}$
 - Via s_4 (since $a \bmod 2 = 0$): $V(s_4) = \{a:\{5\}, b:\{3\}\}$
 - Via s_5 : $V(s_5) = \{a:\{5\}, b:\{2\}\}$
 - Return to s_0 : $V(s_0) = \{a:\{5\}, b:\{3\}\} \quad \{a:\{6\}, b:\{2\}\} = \{a:\{5,6\}, b:\{2,3\}\}$
- Second iteration: Recompute with the new $V(s_0)$. We get $V(s_0) = \{a:\{2,3,5,6\}, b:\{1,2,3\}\}$.
- Third iteration: $V(s_0) = \{a:\{0,1,2,3,5,6\}, b:\{0,1,2,3\}\}$.
- Fourth iteration: $V(s_0) = \{a:\{0,1,2,3,5,6\}, b:\{-1,0,1,2,3\}\}$.
- Fifth iteration: Sets continue to expand (appearance of negative values). We conclude that the loop does not terminate and values diverge.

Example 4: Abstraction by Intervals

Context: Take the same system, but abstract value sets with intervals.

Application: Re-express equations with intervals.

- Initialization: $V(s_0) = \{a:[5,5], b:[3,3]\}$
- First iteration:
 - $V(s_2) = \{a:[5,5], b:[3,3]\}$
 - $V(s_4) = \{a:[5,5], b:[3,3]\}$
 - $V(s_5) = \{a:[5,5], b:[2,2]\}$
 - $V(s_0) = \{a:[5,5], b:[3,3]\} \quad \{a:[6,6], b:[2,2]\} = \{a:[5,6], b:[2,3]\}$
- Second iteration: We get $V(s_0) = \{a:[2,6], b:[1,3]\}$.
- Third iteration: $V(s_0) = \{a:[0,6], b:[0,3]\}$.
- Fourth iteration: $V(s_0) = \{a:[0,6], b:[-1,3]\}$.
- Fifth iteration: $V(s_0) = \{a:[-6,6], b:[-2,3]\}$ (approximating appeared negative values). The interval for a expands toward negatives, and we can conjecture it will become $(-\infty, +\infty)$ after enough iterations.

Example 5: Abstraction by Signs

Context: Now use sign abstraction.

Application: Translate values to signs.

- Initialization: $V(s_0) = \{a:>0, b:>0\}$ (since $5>0, 3>0$)
- First iteration:
 - $V(s_2) = \{a:>0, b:>0\}$
 - $V(s_4) = \{a:>0, b:>0\}$
 - $V(s_5) = \{a:>0, b: 0\}$ (since $b-1$ can be $2>0$, but we over-approximate by 0)
 - Return to s_0 : Union of signs: $\{a:>0, b:>0\} \cup \{a:>0, b: 0\} = \{a:>0, b: \}$ (since $>0 \cup 0 = \text{for } b$)
- Second iteration: With $b: \text{ }$, abstract calculations will yield very broad signs. Quickly, we get $V(s_0) = \{a: \text{ }, b: \text{ }\}$, meaning any value is possible for a and b . This indicates that sign analysis is too coarse for this program, but it allows concluding that the program may not terminate (since we cannot guarantee that a becomes 0).

Synthesis

Abstract interpretation is a static analysis technique that approximates program behavior without concretely executing it. First construct a CFG, then define a collecting semantics equation system. Iterative resolution of this system may not terminate on complex programs. To guarantee termination, we use abstract domains (intervals, signs) that simplify data and allow computing a fixed point in a finite number of steps. However, abstraction causes loss of precision: we can obtain safe results but sometimes too broad.